

TECHNICAL FRAMEWORK DOCUMENT

Assistive Orchestration

A continuous, state-aware dependency scheduler for voice-controlled multi-device assistive environments.

6,347

LINES, APPLICATION

495

AUTOMATED TESTS

11

SUPPORTED INTENTS

16

DEVICES / 3 NODES

Architecture, scheduling model, execution semantics, safety envelope and recovery behaviour.
Generated from the implemented system — all figures measured, not estimated.

01 What the system is

A person speaks. The system decides what has to happen in the physical environment to satisfy that request *from where things currently stand*, then carries it out across distributed devices and records only what the hardware confirmed.

The distinguishing property is in the second step. A conventional voice-controlled home maps an utterance to a fixed command or a fixed script. This system treats the utterance as a *goal* and derives the action sequence from the current logical world — so the same sentence produces different work depending on the state of the house.

The claim, stated precisely

```
# A fixed-script system
intent -----> fixed action list

# This system
intent + current state + topology + device dependencies --> required action list
```

Measured across the implemented system: **48 (intent, starting-state) pairs produce 43 distinct action sequences**. A fixed-script design over the same 8 movement intents would produce 8.

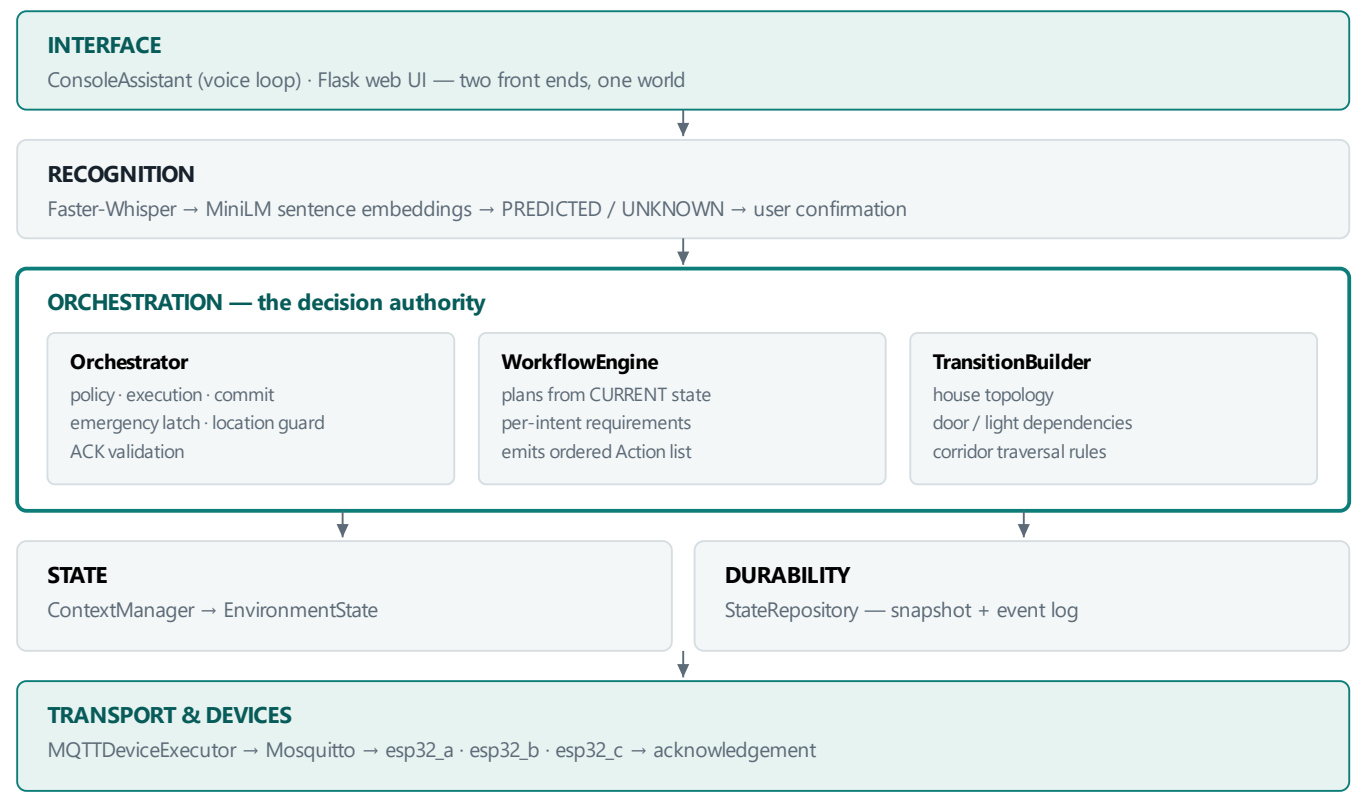
Scope and honesty

COMPONENT	STATUS
Speech → text, intent recognition, confirmation	IMPLEMENTED
State model, topology, workflow generation	IMPLEMENTED
MQTT transport, acknowledgement, retry, idempotency	IMPLEMENTED
Emergency lifecycle, persistence, restart recovery	IMPLEMENTED
ESP32 device nodes	SIMULATED — three mock node processes; no physical hardware
Physical-state sensing / reconciliation	NOT PRESENT — no sensors of any kind
Comparative experimental evaluation	NOT PRESENT — correctness tests only

Read this before citing the document

The nodes are software processes that simulate hardware. Every device state in this system is what the software *believes* after a confirmed command, never a reading from a sensor. That distinction is load-bearing throughout the design and is why the uncertainty model in §6 exists.

02 Layered architecture

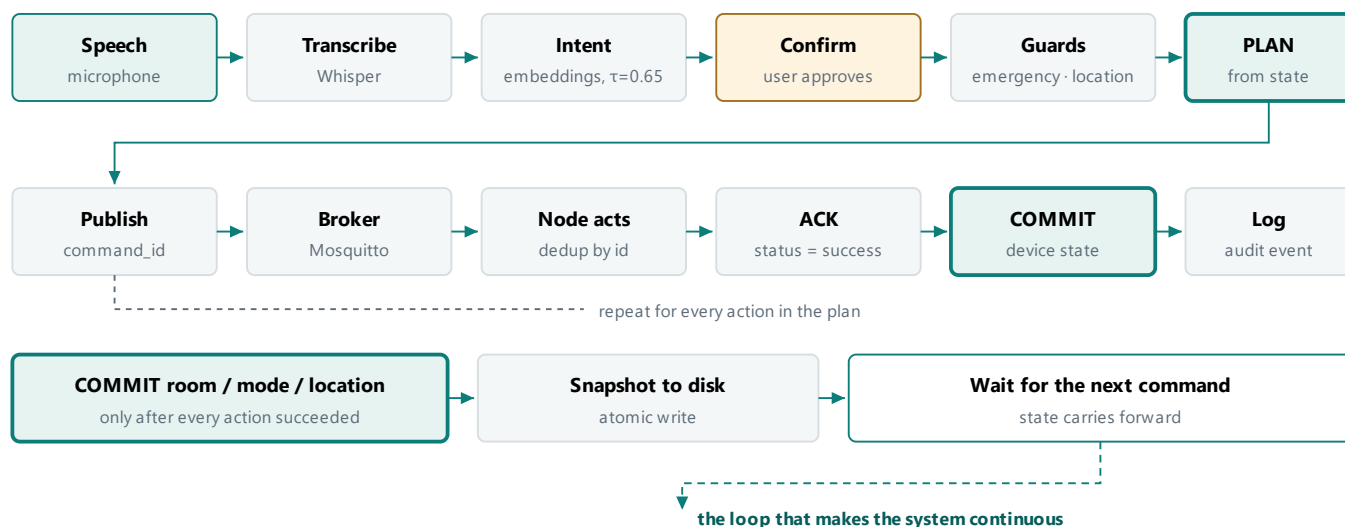


Each layer has one responsibility. Planning never touches transport; transport never touches state; the interface never builds an action.

Responsibility boundaries

COMPONENT	OWNS	EXPLICITLY DOES NOT
Orchestrator	Policy, execution order, ACK validation, all state commits	Build plans; know MQTT exists
WorkflowEngine	What a goal requires given the current state	Execute; mutate state
TransitionBuilder	Topology and movement dependencies	Know about intents
ContextManager	The authoritative logical state	Know about files or storage
MQTTDeviceExecutor	Publish, correlate, retry, await ACK	Own any workflow state
StateRepository	Atomic snapshots, append-only event log	Make any decision
Mock ESP32 node	Simulated pins, command dedup, status replies	Know about rooms, intents or the world model

03 The command pipeline



The pipeline. The dashed return path is the property that distinguishes this from a one-shot assistant: the world that command n commits is the world command $n+1$ plans against.

04 The dependency scheduler

This is the core of the framework. "Scheduler" here means dependency resolution, not time: nothing in the system is triggered by a clock, a calendar or a timer.

4.1 The scheduling function

$$W = f(I, S, T, D, P)$$

I	intent	one of 11 supported goals
S	current logical state	room, mode, location certainty, every device
T	topology	which rooms connect, through what
D	device dependencies	a room must be lit before it is used, a door opened before it is passed, a room closed on exit
P	policy	emergency latch, location certainty guard
W	ordered action sequence	

And the state advances only on confirmation:

$$S(t+1) = F(S(t), W, ACK) \quad \# \text{ no ACK, no state change}$$

What this is, technically

A hand-specified, state-conditioned transition expander. There is no search, no goal representation and no declarative precondition language — preconditions are expressed as conditionals over the state, and ordering is implicit in the emitted sequence. It is closer to a parameterised finite-state machine than to a classical task planner, and the document describes it that way deliberately.

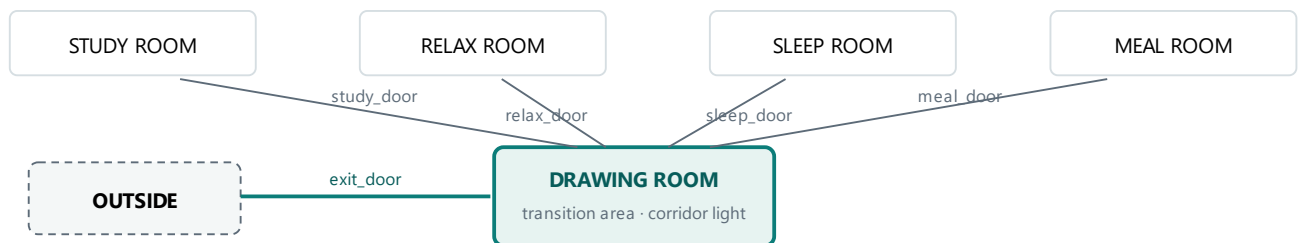
4.2 Worked example — one intent, six starting states

The intent is identical in every row below: `PREPARE_FOR_SLEEP`. Only `S` changes.

STARTING STATE	w	GENERATED ACTION SEQUENCE
SLEEP_ROOM already there	2	LIGHT_ON sleep_light · PREPARE_BED sleep_bed
DRAWING_ROOM in the corridor	6	LIGHT_ON drawing_light · OPEN sleep_door · CLOSE sleep_door · LIGHT_ON sleep_light · PREPARE_BED · LIGHT_OFF drawing_light
OUTSIDE entering the house	8	OPEN exit_door · CLOSE exit_door · LIGHT_ON drawing_light · OPEN sleep_door · CLOSE sleep_door · LIGHT_ON sleep_light · PREPARE_BED · LIGHT_OFF drawing_light
STUDY_ROOM	9	LIGHT_OFF study_light · OPEN/CLOSE study_door · LIGHT_ON drawing_light · OPEN/CLOSE sleep_door · LIGHT_ON sleep_light · PREPARE_BED · LIGHT_OFF drawing_light
MEAL_ROOM	9	as above, via meal_door
RELAX_ROOM has an extra device	10	LIGHT_OFF relax_light · TV_OFF relax_tv · OPEN/CLOSE relax_door · LIGHT_ON drawing_light · OPEN/CLOSE sleep_door · LIGHT_ON sleep_light · PREPARE_BED · LIGHT_OFF drawing_light

Three dependency behaviours are visible in that table alone: the corridor light is raised only when the corridor is actually traversed; the source room is shut down before it is left; and the relax room contributes an extra action because it owns a television.

4.3 Topology



Every functional room is reached through the Drawing Room.

There is no direct study→sleep edge: that journey is two traversals and a lit corridor.

The topology is a star. It is declared once in `TransitionBuilder` and is the only place in the system that knows how the house is laid out.

4.4 The corridor invariant

The most instructive dependency rule in the system, because it is conditional in both directions:

- The corridor light is raised **before** the destination door opens — and only if the corridor is genuinely traversed.
- It is lowered **after** arrival — and only if it was raised.
- If the person is already in the destination, no corridor action is emitted at all.
- If the person is standing *in* the corridor and it is already lit, no redundant command is sent.

All 6×4 combinations of source and destination are enforced by 120 automated tests.

05 State model & commit semantics

5.1 What the state holds

FIELD	MEANING
current_room	Last committed location of the person
location_status	KNOWN · IN_TRANSIT · UNKNOWN — how far current_room can be trusted
location_destination	Where an in-progress move is heading
location_reason	Why certainty was lost, for the operator
current_mode	NONE · STUDY · RELAX · SLEEP · MEAL · EMERGENCY
return_target	The room to come back to
emergency_declared_at / _cleared_at	Durable emergency lifecycle record
drawing_light, exit_door, buzzer	Shared devices
study / relax / sleep / meal	Per-room device states (13 fields)

5.2 Two commit tiers

The system commits at two different granularities, and the distinction matters for correctness under failure.

Per acknowledged action

Device state. A light is recorded ON the moment its node confirms it. Ten actions, ten small commits.

Per completed workflow

Room, mode, return target, location. Committed only once *every* action in the plan has been acknowledged.

The consequence is deliberate: a workflow that fails part way leaves the devices it actually moved recorded as moved, and does **not** claim the person arrived.

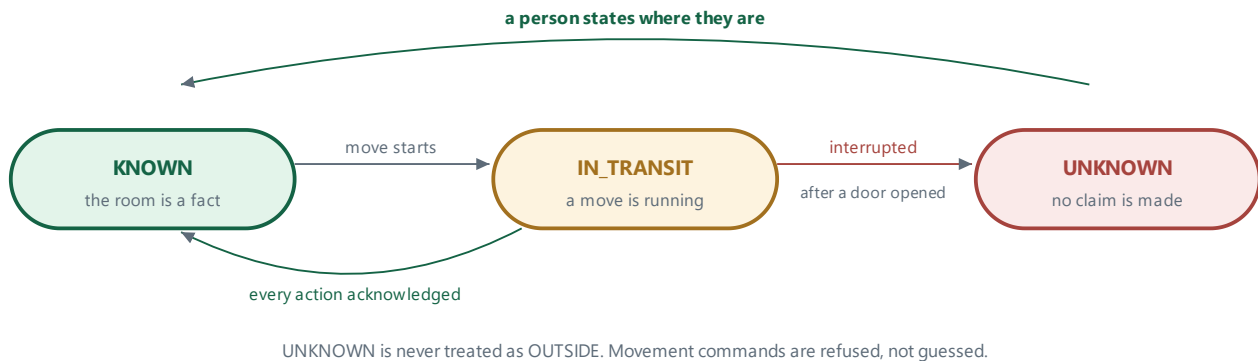
5.3 What counts as an acknowledgement

```
# All four must hold before any state moves
result is a mapping
result["status"] == "success"
result["device"] == the device we commanded
result["action"] == the action we sent
```

A timeout, a device-reported error, a malformed reply, or a reply about a different device all leave the logical state untouched.

06 Location certainty

The system has no sensors. It cannot see where the person is; it can only remember where it last successfully sent them. This section is about being honest about that.



Uncertainty is modelled as a property of the reading, not as a place. It is deliberately not a member of the Room enumeration.

The interruption rule

When a movement workflow fails part way, the system decides what it can still honestly claim using a physical argument:

No door was opened

The person cannot have left. The source room is still true, so the location stays **KNOWN**. A failed light does not trigger a dialogue.

A door was opened

They may be anywhere along the route. The only honest answer is **UNKNOWN**, and movement commands are refused until a human resolves it.

Why this matters more once state is durable

Without persistence a stale location lives only as long as the process. Once state survives a restart, a location the system cannot vouch for becomes a durable false claim — and every future plan would be built on it.

07 Reliability & safety envelope

7.1 Exactly-once physical execution

A retry exists because an acknowledgement did not arrive — not because the action did not happen. The system is built around that distinction.

```
# One logical action = one identity, reused across all attempts
attempt 1 -> command_id = 4b45c942... (no ACK)
attempt 2 -> command_id = 4b45c942... (node recognises it)
attempt 3 -> command_id = 4b45c942...

# Node side: 128-entry history of completed command ids
duplicate id -> re-acknowledge, do NOT actuate
```

Verified end-to-end against a live broker: an acknowledgement was dropped, the executor retried, and the door opened **once**.

7.2 Emergency lifecycle

STAGE	BEHAVIOUR
Entry	Mode latched <i>before</i> any hardware is touched — a partly failed alarm is still an emergency. Buzzer raised, exit and all internal doors opened.
Held open	Nothing closes the escape route. Entry emits no CLOSE_DOOR at all.
While latched	All ordinary intents refused. Only EMERGENCY (idempotent retry) and EMERGENCY_CLEAR are accepted.
Best effort	A failed door does not abandon the remaining doors; failures are reported, not swallowed.
Clear	Confirmed command silences the buzzer and closes up. If any required action fails, the latch <i>holds</i> .
Across restart	Latch restored from durable evidence. No actuator is re-fired — a stored file is not proof the buzzer is sounding.

A design decision worth stating explicitly

Entry deliberately does *not* close the doors it opens. In an earlier revision it did, and the escape route was measured open for 7 milliseconds. The software cannot know whether anyone got out, so the route now stays open until a human says otherwise.

7.3 Medication

Twelve of the thirteen action types are set-to-a-state operations and converge on repetition. ACTIVATE_MEDICATION is an impulse — it dispenses a dose. It is recorded in the audit log and **never replayed**; no replay mechanism exists anywhere in the system.

08 Persistence & restart recovery

8.1 What is stored

Snapshot — state.json

The whole world plus metadata. Written atomically: temp file → flush → fsync → `os.replace`. Previous copy kept as `.bak`.

Event log — events.jsonl

One line per acknowledged action: timestamp, intent, device, action, command_id, node, status, resulting state. An audit trail, never a replay queue.

```
{ "schema_version": 1,
  "application_version": "0.10.0",
  "saved_at": "2026-08-27T04:49:49Z",
  "clean_shutdown": false,
  "state": { ... the full environment ... } }
```

8.2 How far a restart trusts what it finds

STORED STATUS	PREVIOUS EXIT	RESTORED AS	REASONING
KNOWN	clean	KNOWN	Nothing happened in between that could have been missed
KNOWN	crash, nothing committed	KNOWN	The stored world is identical to a fresh one — no claim to distrust
KNOWN	crash, after real work	UNKNOWN	The session ended mid-life; the location may be stale
IN_TRANSIT	any	UNKNOWN	The move never completed; both endpoints retained for diagnosis
UNKNOWN	any	UNKNOWN	Uncertainty is never laundered into a room

The clean-shutdown marker is written *dirty at startup* and set true only by an orderly shutdown, so an abandoned process cannot masquerade as a clean exit.

The defect this closed

Before persistence, restarting the application silently cleared an active emergency: a fresh state object reported mode NONE and accepted ordinary commands again. A restart was, in effect, an unauthenticated emergency clear.

09 Evidence & limitations

9.1 Test coverage

SUITE	TESTS	WHAT IT PINS
test_transition_invariants	120	All 6×4 source/destination pairs, corridor invariant
test_emergency_lifecycle	50	Latch, blocking, clear, failure semantics
test_emergency_recognition	43	Safety-critical recall and false-alarm prevention
test_state_persistence	40	Round-trip, atomicity, corruption rejection
test_location_semantics	39	KNOWN / IN_TRANSIT / UNKNOWN behaviour
test_startup_recovery	36	Crash detection, emergency survival, no replay
test_dataset_hygiene	26	Corpus integrity, leakage, threshold consistency
others (11 suites)	141	MQTT end-to-end, idempotency, parity, web flow, runtime
Total	495	493 pass, 2 skipped, 0 failures

9.2 Recognition performance

Measured on a 110-sentence holdout with verified zero overlap against the training corpus:

METRIC	BEFORE	AFTER
EMERGENCY recall	15%	90%
EMERGENCY false positives	1	0
EMERGENCY_CLEAR recall	80%	100%
Ordinary-intent accuracy	47%	68%

Every remaining holdout failure resolves to UNKNOWN rather than to a wrong workflow — the system asks the user to repeat, it does not act on a misreading.

9.3 Limitations — stated plainly

No physical hardware

All three nodes are software. Nothing in this document has been demonstrated on a real ESP32 or a real door.

No comparative evaluation

495 tests establish correctness against a specification. They do not establish that the specification is better than an alternative design.

No sensing

All 16 devices are actuators. Logical state is what the software believes after a confirmed command, never a measurement.

Single writer

Two application instances sharing one state directory would interleave writes. This is documented, not solved.

9.4 What the framework does not claim

MQTT, ESP32, publish/subscribe, speech-to-text, sentence embeddings, retries, command identifiers, write-ahead logging and atomic file replacement are all standard, established technology. The framework's contribution — if any — lies in how the state, uncertainty and safety semantics are composed around them, and that composition has not yet been evaluated against a baseline.